Unity Vive Tracker System

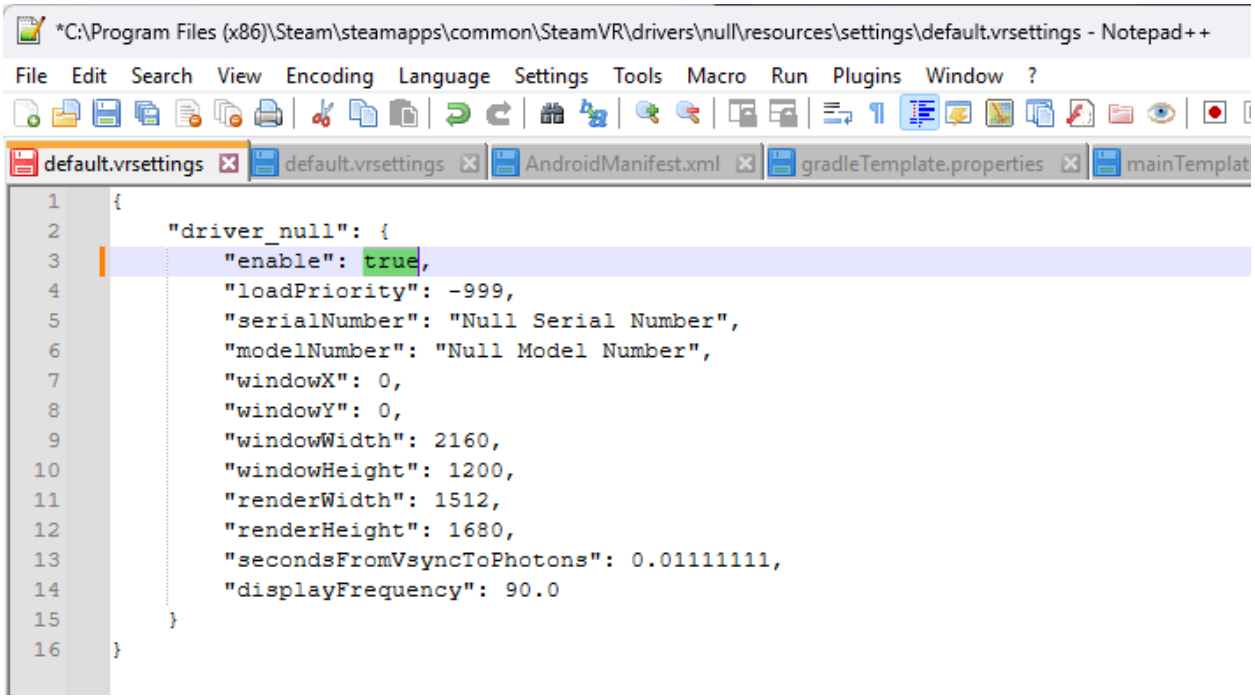
SteamVR Settings

Null Driver

The first step is to install SteamVR on the PC and then change the SteamVR settings so that the trackers can work without a headset.

The file can be found in the path below:
C:\Program Files (x86)\Steam\steamapps\common\SteamVR\drivers\null\resources\settings\default.vrsettings

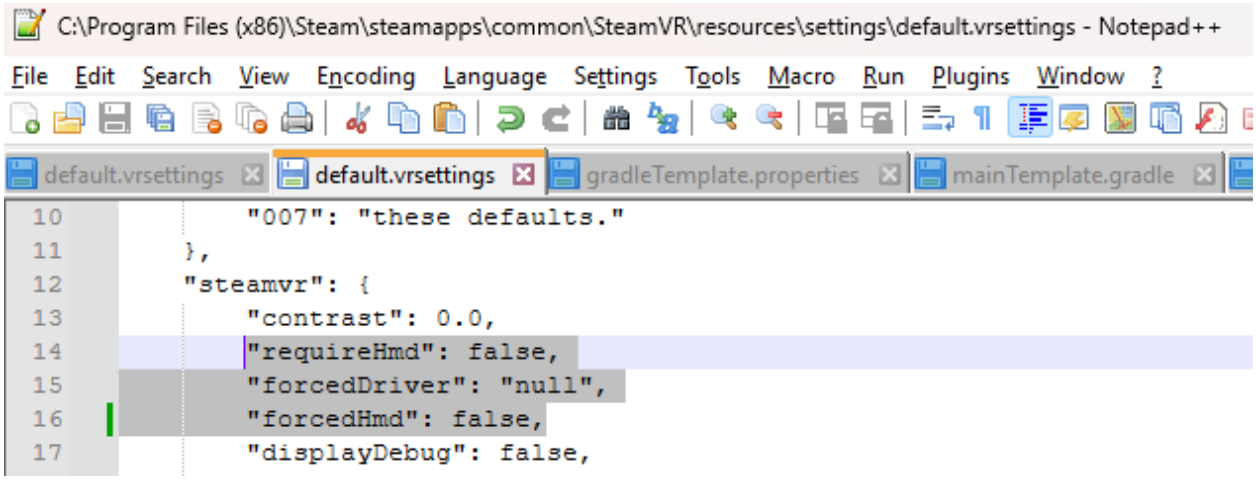
We need to set “enable” to true. This will tell steamVR that we want to use it without a headset.



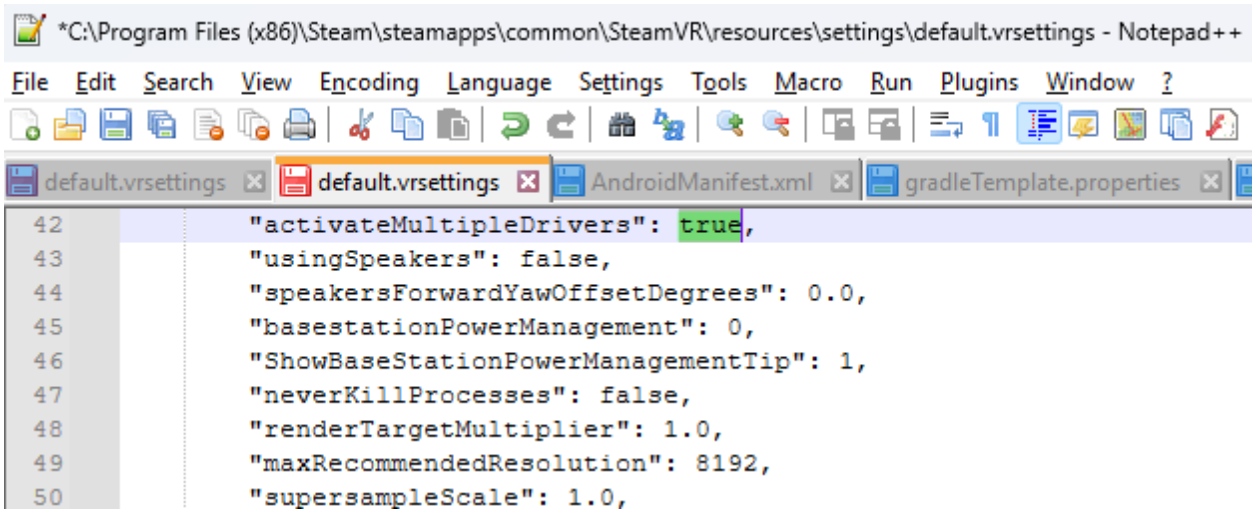
Default Settings File

Another file that we need to change is found in the path below:
C:\Program Files (x86)\Steam\steamapps\common\SteamVR\resources\settings\default.vrsettings

Set “requireHmd” to false
Set ‘forcedDriver” to “null”
Set “forcedHmd” to false



Set “activateMultipleDrivers” to true



Now restart SteamVR and connect the Vive Trackers to it.

Hardware Installation

The Vive trackers need to be turned on in order for the base station to see them.

Here is a guide with how to connect the dongles to the pc.
https://www.vive.com/us/support/tracker3/category_howto/pairing-vive-tracker.html

Here is a guide on how to install the base stations:
https://www.vive.com/us/support/vive-pro/category_howto/installing-the-base-stations.html

The base stations need to be on a tall object or a support that has direct view to the Vive Trackers. If none of the base stations see the Vive Trackers at one point, the tracking will stop until the line of sight is valid again.

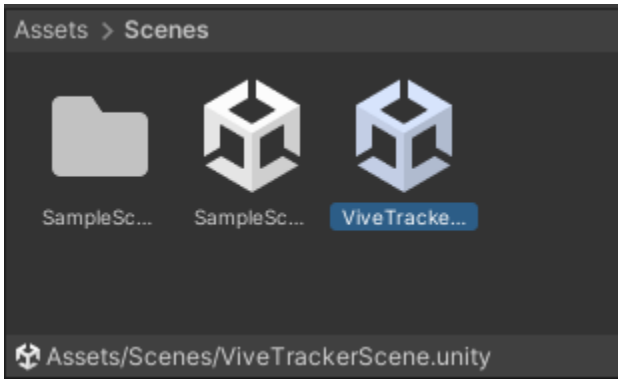
The vive trackers are the main device that communicated with the PC, the base stations are only shooting laser lines in the rooms to be picked by the Vive Trackers. So the base stations do not connect to the PC, they only need to be connected to the power supply, and the Vive Trackers will give the information to SteamVR about their existence.

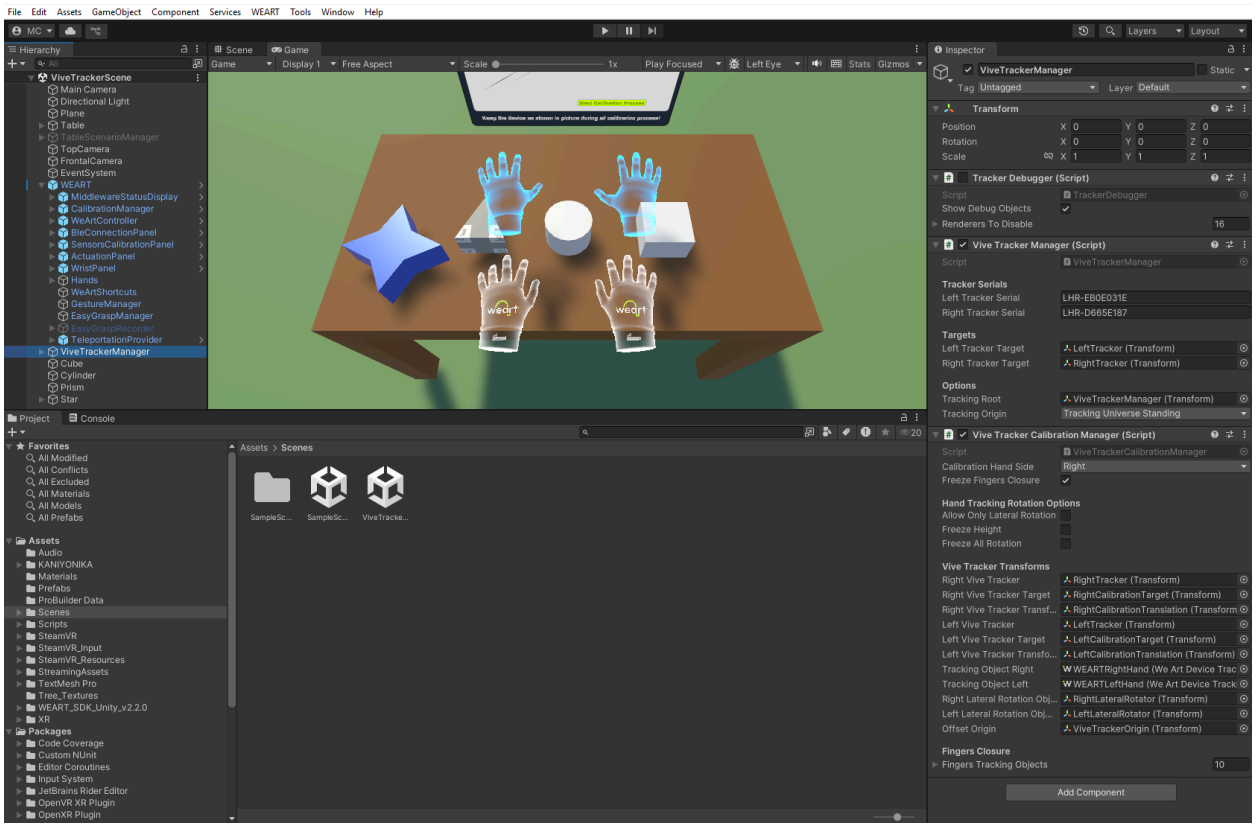
The Vive Tracker dongles need to be connected to the PC so that the Vive Trackers can communicate with the PC.

Unity Project Overview

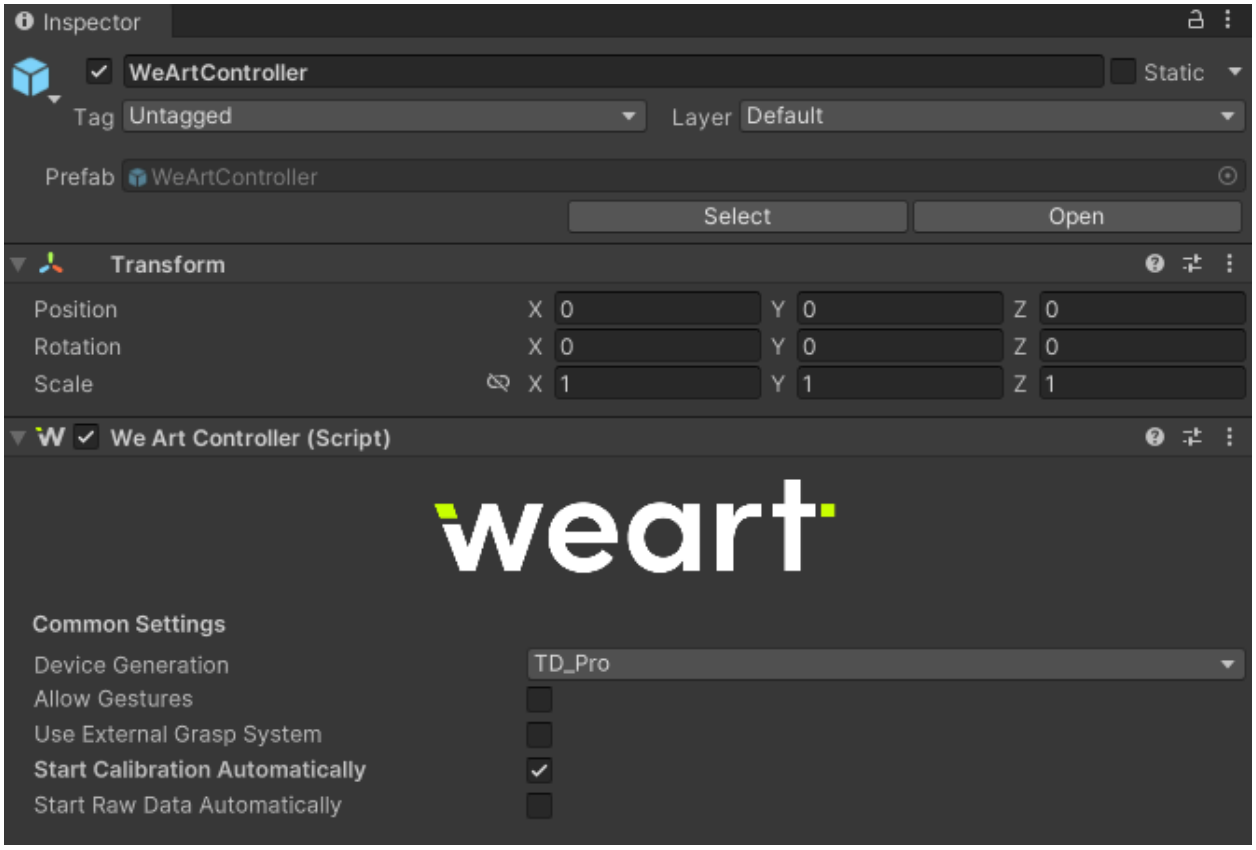
Scene

The scene can be found here:



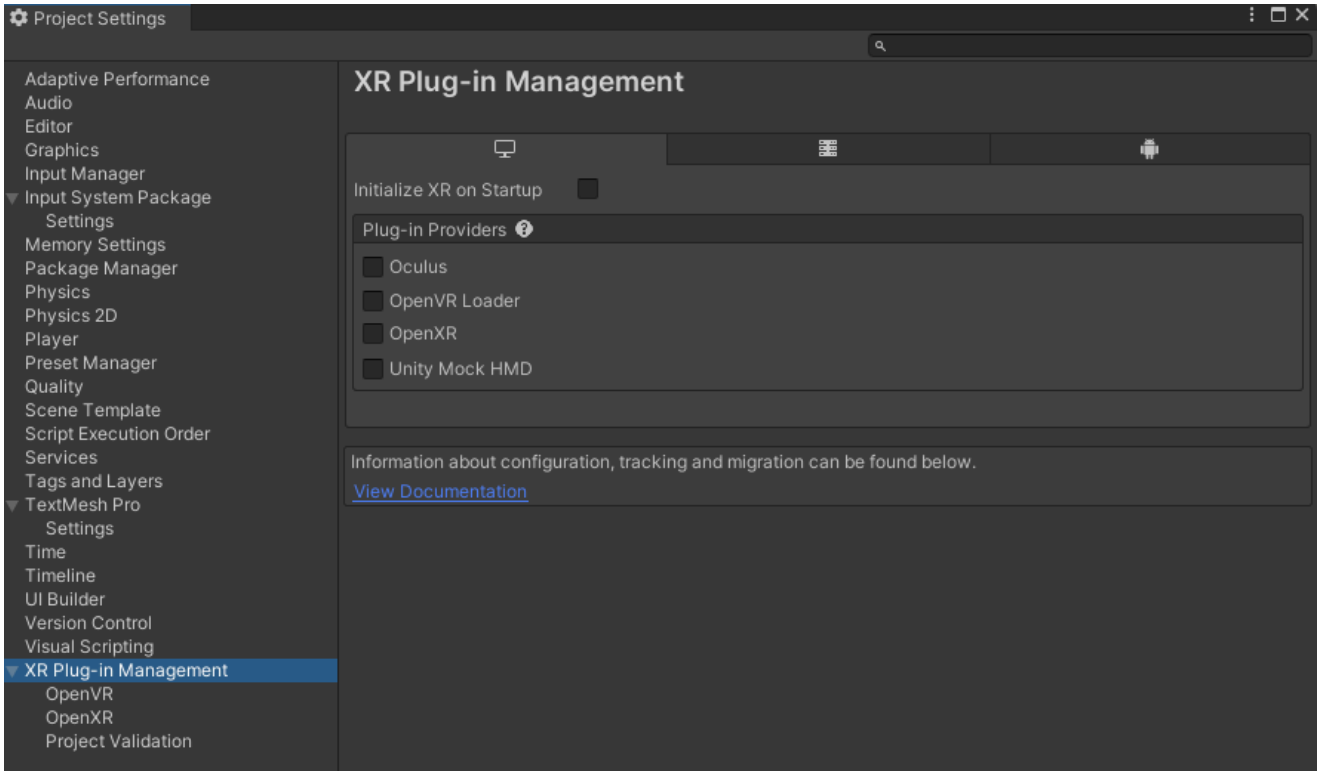


WeArtController is set to StartCalibrationAutomatically, this means that on scene start, it will calibrate the Touch Divers and allow the WeArt App to send information to the touch divers.



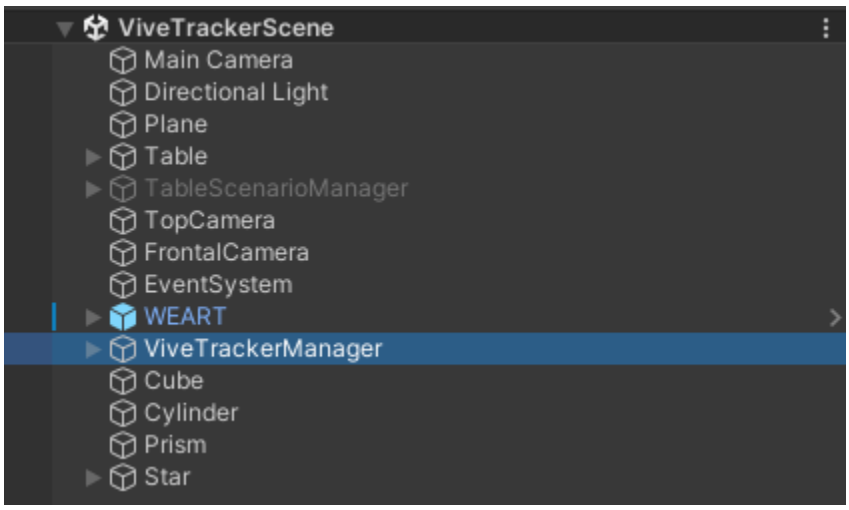
Project Settings

These are the project settings needed to enable the Vive Trackers without a headset. The project already has them set.



ViveTrackerManager

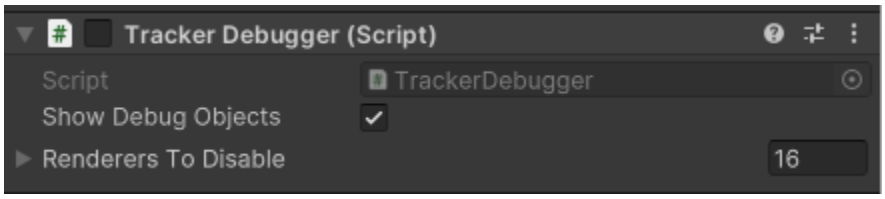
This game object contains three main scripts used for tracking.



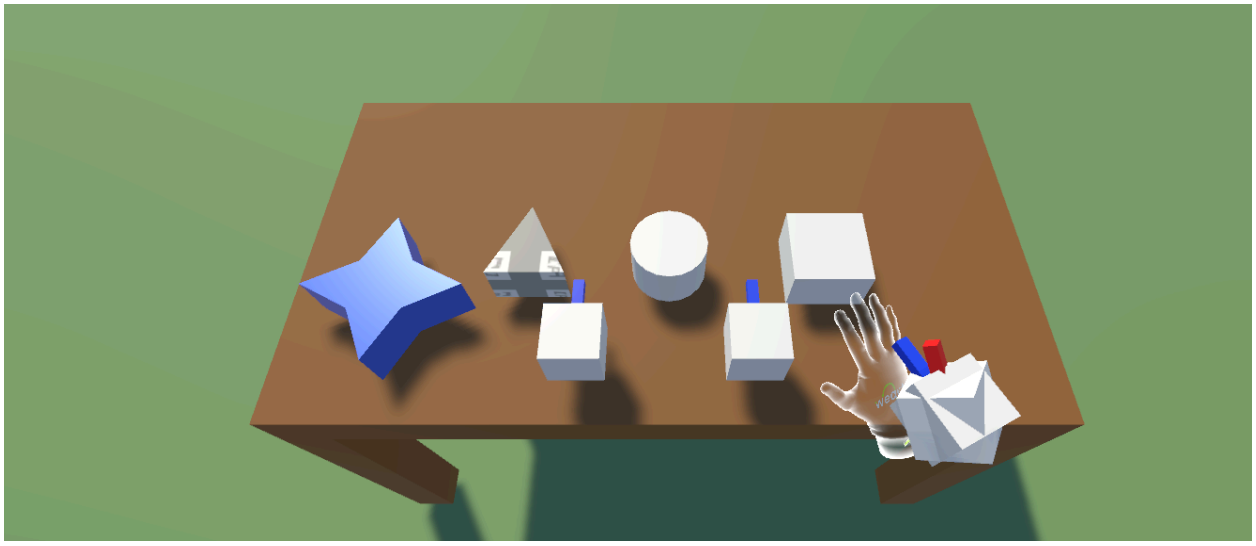
Tracker Debugger

When enabled as a component, it will read all Vive Trackers connected to steamVR and it will continuously debug.log the serial number of the ViveTracker and its position.

This is useful because we can differentiate the right from the left tracker and get the serial numbers.



When Show Debug Objects is enabled, when the scene starts it will show the gameobjects used for calculating the tracking.



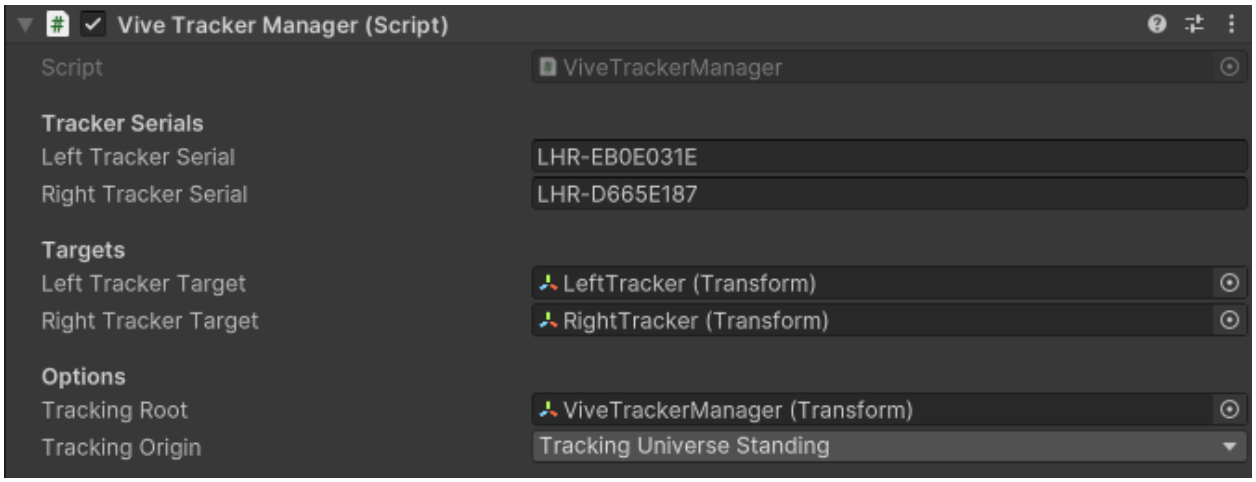
The red object displays the real ViveTracker position and rotation.

The Two cubes on the table represent the calibration reference, these cubes determine the standard height and direction for calibration.

The cubes on the table are called RightCalibrationTarget and LeftCalibrationTarget

They ensure that when pressing “space” on the keyboard, the hands will align correctly them, creating a good calibration

Vive Tracker Manager

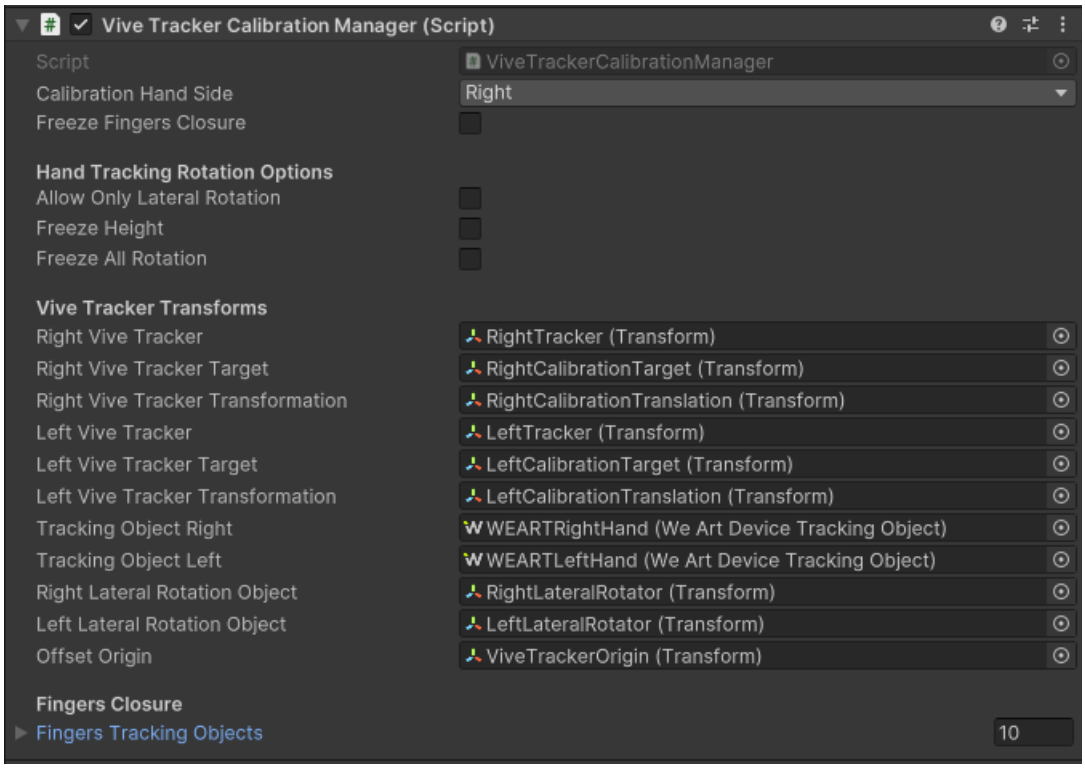


This Script is responsive for reading the position and rotation from the vive trackers.

With the Serial number taken form TrackerDebugger script(this is done by enabling it and seeing the serial numbers that are displayed in the console using Debug.Log) We can take these serials and put them in the left and right tracker serial fields, corresponding to the correct hand sides.

Vive Tracker Calibration Manager

This script is responsible for the calibration of the steamVR space to our own coordinate system.



When pressing “space” on our keyboard, this script will make sure that the Right Vive Tracker is aligned to RightCalibrationTarget (this can be changed to left hand if needed with CalibrationHandSide field)

This script also contains four other fields:

- FreezeFingersClosure (makes the hands ignore the closure and abduction values for fingers, making the fingers never bend)

Rotation Options

- AllowOnlyLateralRotation (makes the hand only be able to rotate on the y axis)
- FreezeHeight (does not allow the hand to move up or down)
- FreezeAllRotation (forces the hand to face the direction of the calibration)

These rotation options require the previous field to be enabled. Example: FreezeAllRotation needs FreezeHeight and AllowOnlyLateralRotation to be enabled

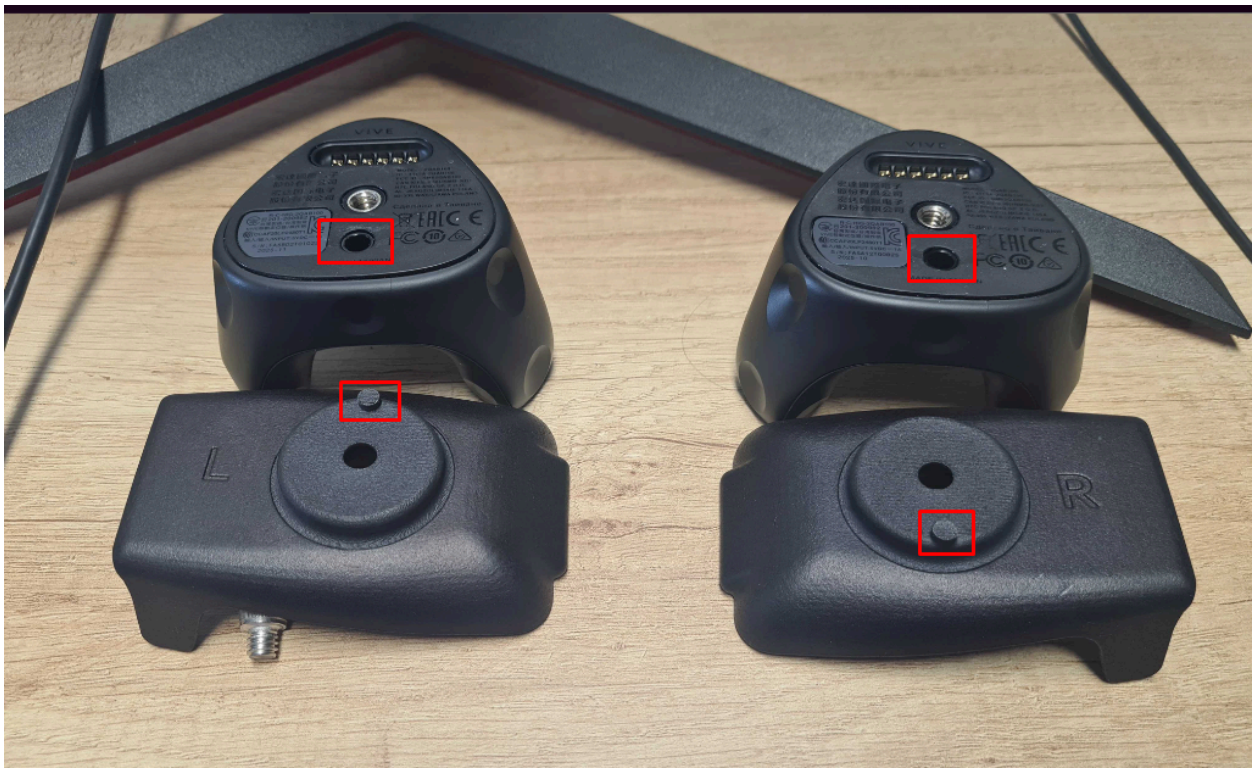
Hardware Setup and Calibration Rules

This is how the Vive Trackers must sit on the TouchDivers:

The Left Vive Tracker must have the green light away from the person using it. It must be in the same direction as the fingers pointing forward.

The Right Vive Tracker must have the green light towards the person using it. It must be in the same direction as the fingers pointing forward.

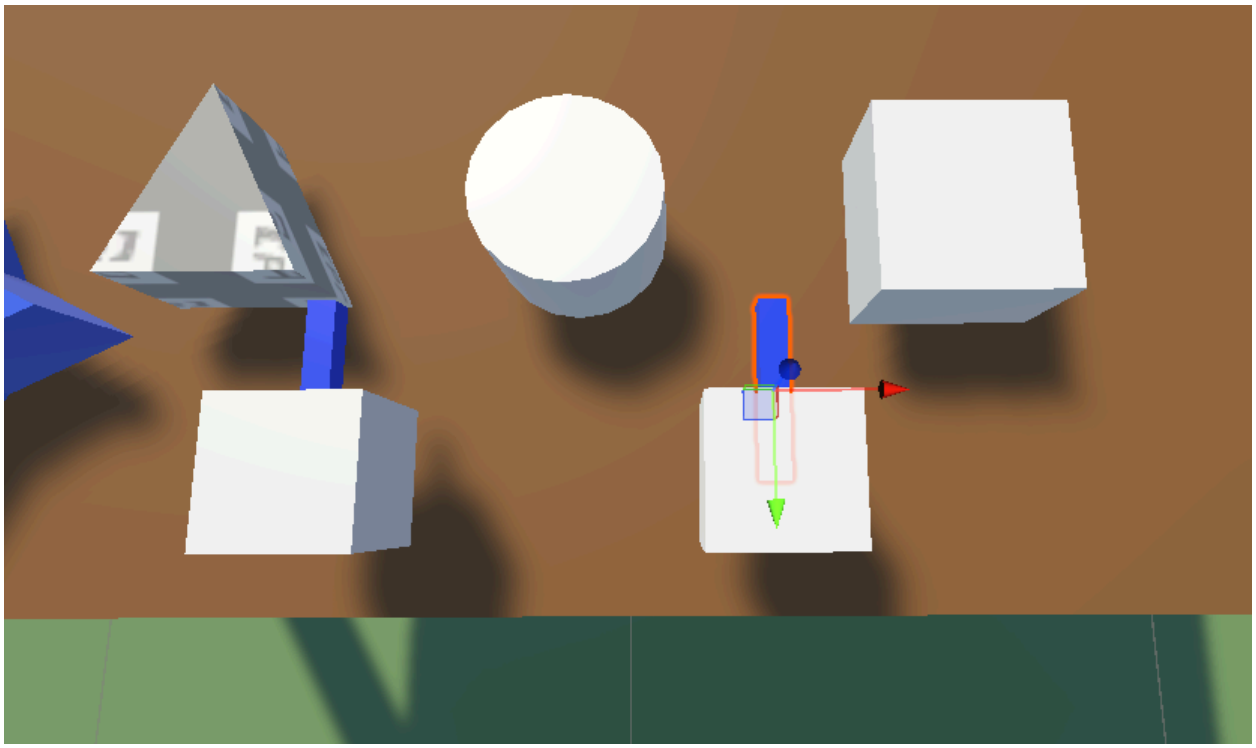
The Vive Tracker holes must be placed on the supports in the corresponding place



Before pressing “space” to calibrate the coordinate systems, make sure the palm of the hand is completely flat on the table. This will ensure the best calibration.

The rotation of the hand is also important. RightCalibrationTarget, is responsible for the initial direction of the hand. In the scene, we can see that the hand needs to be close to the edge of the table facing the person and the hand must be pointing perfectly forward. Making the finger be perpendicular to the edge of the table. Having the hand rotated in the wrong direction will create a bad calibration and it will rotate the whole coordinate system in the wrong direction.

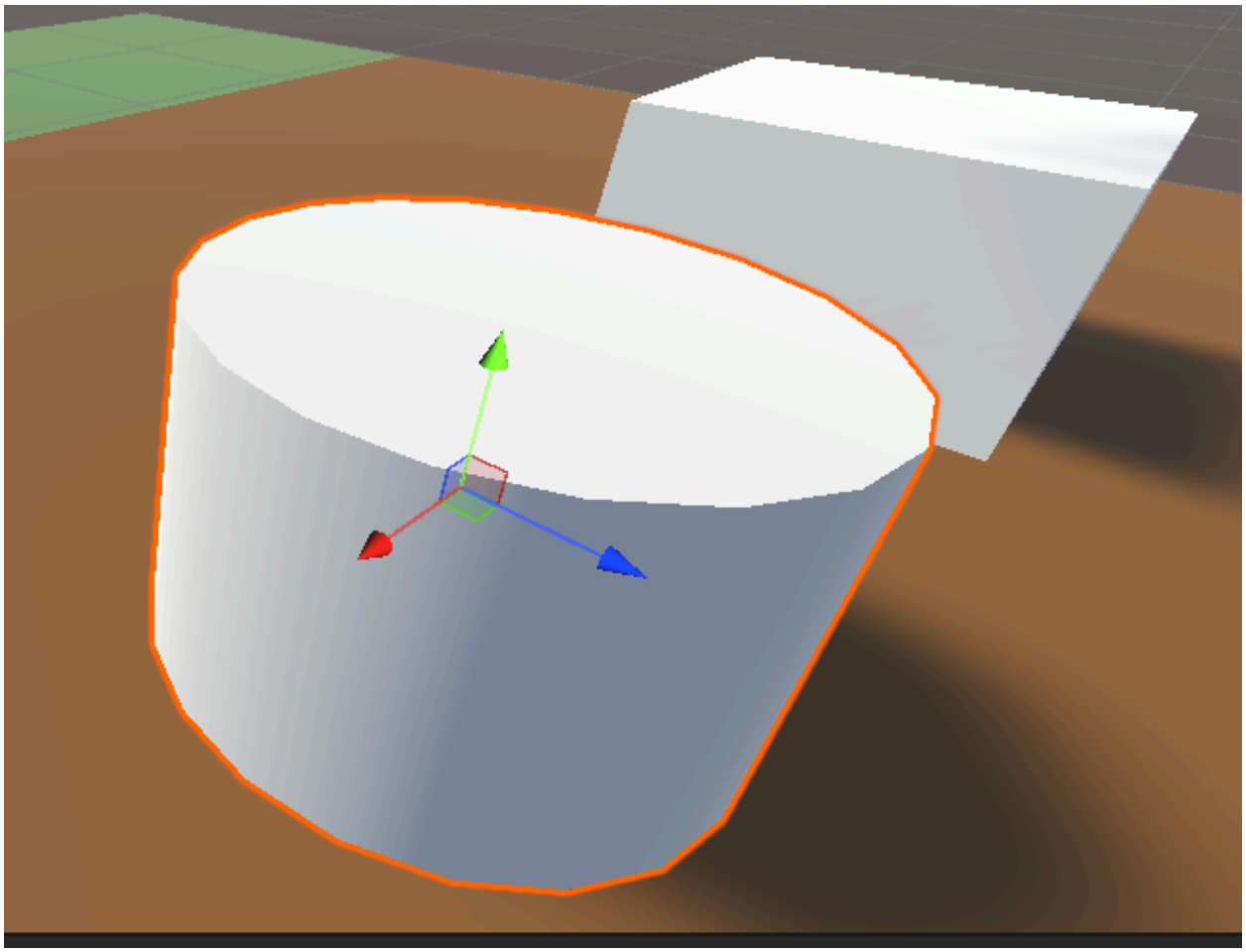
After “space” is pressed on the keyboard, the calibration is complete.

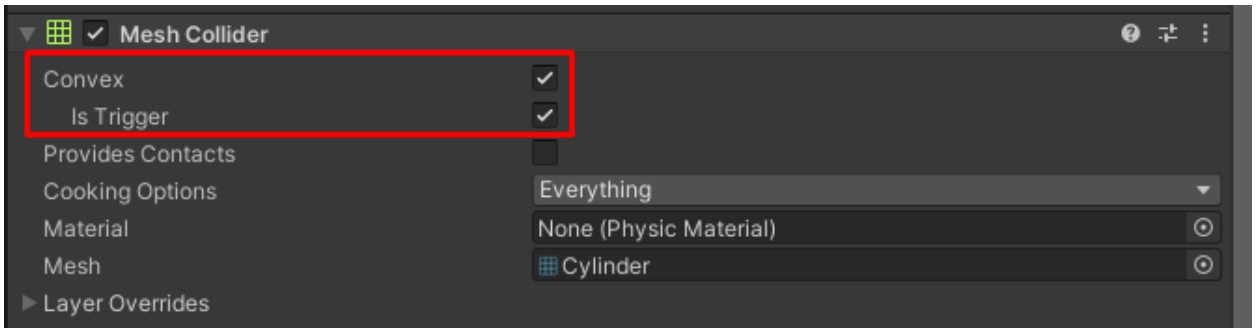


Touchable Objects best practices

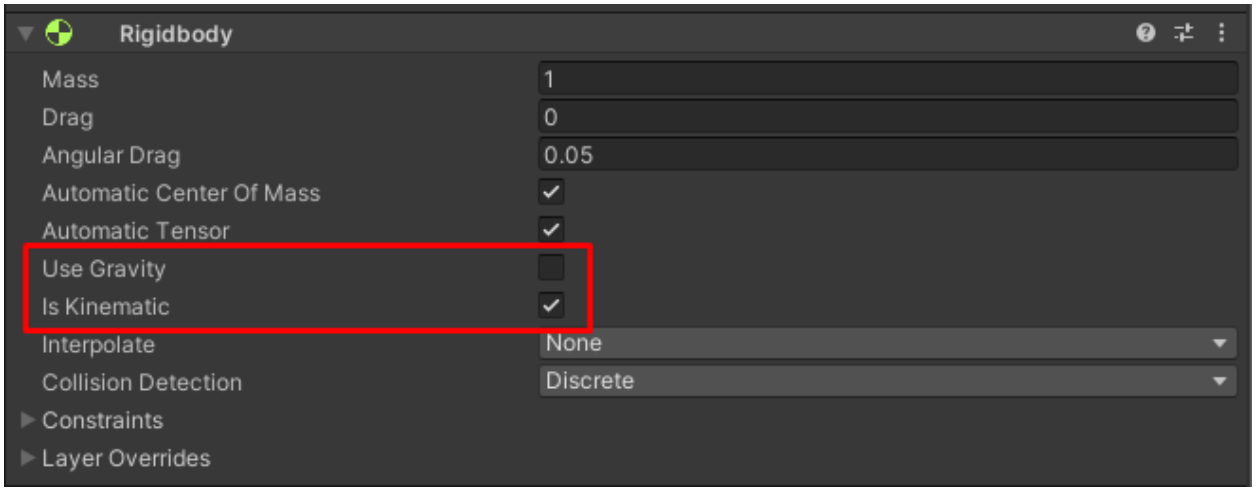
For the purpose of this project here are some options that will give the best results:

For any meshes that are not primitives like cube and sphere the mesh collider must be set to Convex and IsTrigger.
Convex allows for the best interaction with the physics system.
IsTrigger makes the object passable through.



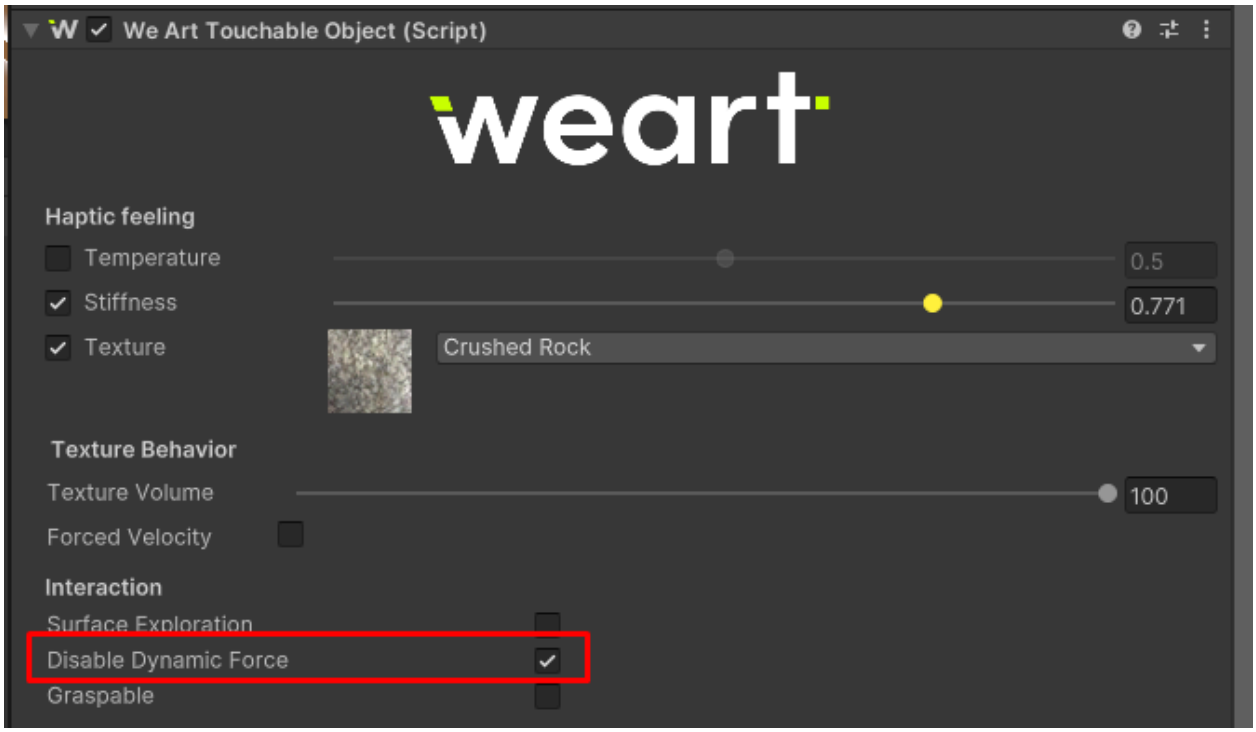


Each object must have a rigidbody attached. And the settings:
Use gravity, disabled, and IsKinematic, enabled these options allow for a floating object.
Rigidbody is required for the physics system to react with haptics for the gloves



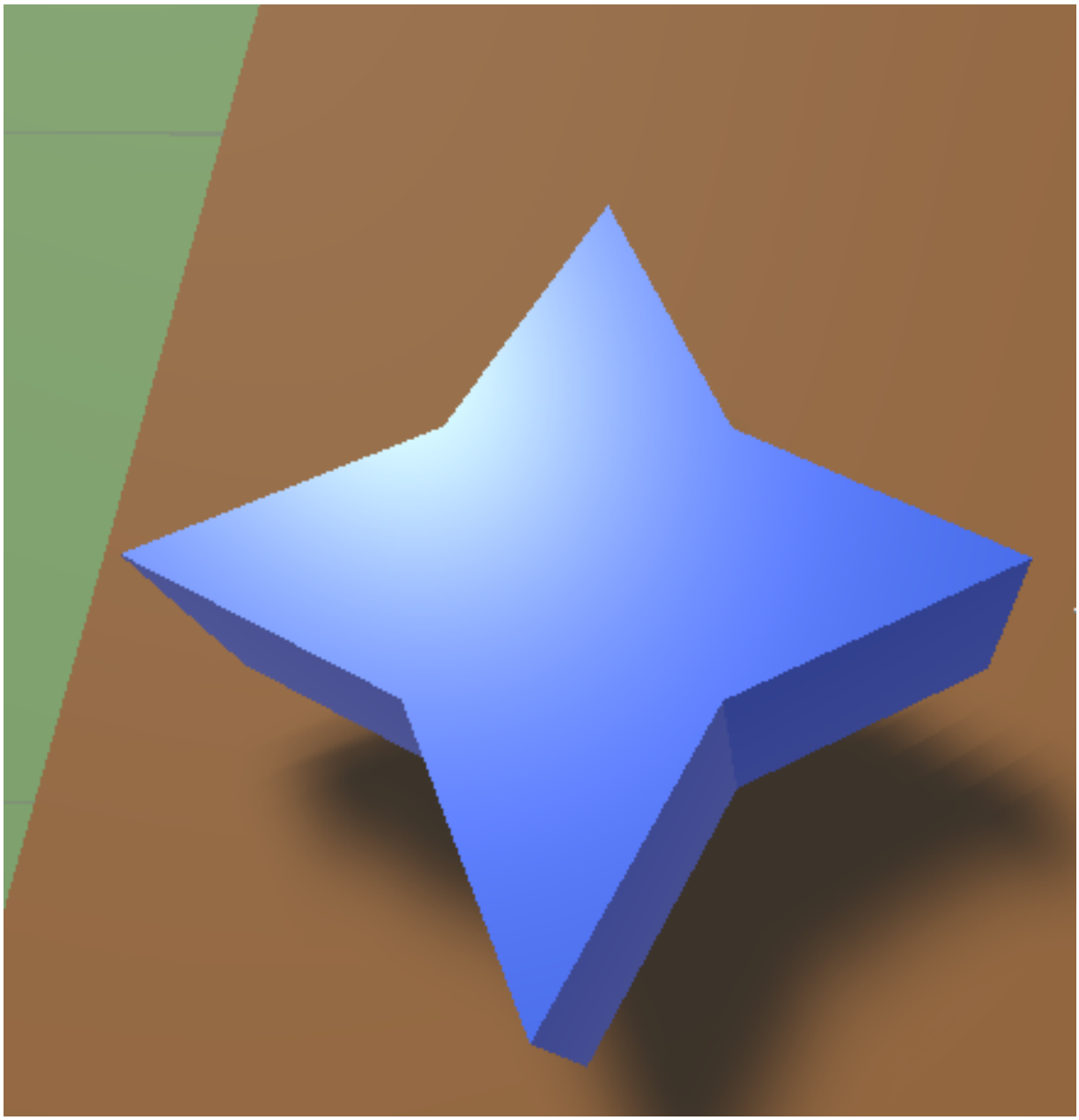
Touchable Object Settings

Make sure to have DisableDynamicForce turned on, it will allow the fingers to feel the direct value of Stiffness. Without this the force will be applied on the fingers in a wrong way.

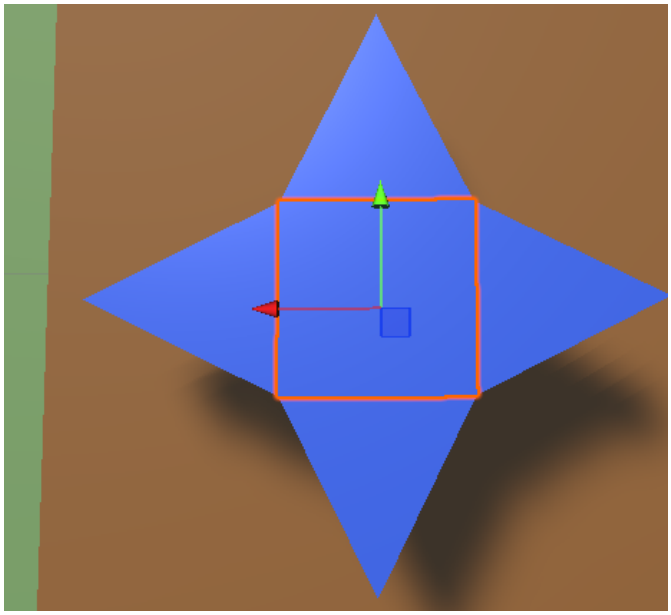


Complex Objects

In the case of a complex object like a star, the mesh must be split in different convex shapes in order to work with the physics system correctly. Each shape must have a Touchable Object and rigidbody.



This was made out of a cube and 4 other convex shapes.



The parent “Star” here is empty and it acts only as a container.

This is how a prism looks, they are all independent objects with their own Rigidbody and Touchable Object. The cube is the same.